

Ucab (GrabMyCab) — All-in-One Documentation

Welcome to **Ucab** (also referred to as *GrabMyCab*), a modern web-based ride-hailing and cab booking simulation platform explicitly focused on the South Indian region (primarily serving cities of **Hyderabad, Telangana** and **Bhimavaram, Andhra Pradesh**).

This documentation provides an all-in-one architectural overview, key pricing/booking business rules, tech stack explanations, directory breakdown, and developer setup instructions.

Tech Stack & Core Libraries

Ucab is built on top of a cutting-edge server-side React architecture:

- **Runtime & Package Manager:** [Bun](#) — powering package resolution, configuration loading, and scripts.
- **Web Framework:** [TanStack Start](#) (React 19 SSR Architecture) — featuring unified full-stack routing, and seamless hydration.
- **Routing System:** [TanStack Router](#) — file-based routing engine providing deep type-safety from parameters to loader payloads.
- **Styling & Design System:** [Tailwind CSS v4](#) — integrated via Vite compile-time plugins (`@tailwindcss/vite`) to manage color systems, layout wrappers, and custom themes in `src/styles.css` .
- **Component Foundation:** [Radix UI](#) primitives & [shadcn/ui](#) configurations (`components.json`).
- **Icons & Bundling:** [Lucide React](#) for dashboard visuals and [Vite](#) to build code assets.

Project Directory Structure

```
grab-my-cab/
├─ .gitignore
├─ .lovable                # Configuration and syncing parameters for Lovable IDE
├─ .prettierignore
├─ .prettierrc
├─ bun.lock                # Bun lockfile for super-speedy dependencies resolution
├─ bunfig.toml             # Bun engine override configurations
├─ components.json         # Design systems config matching shadcn/ui components
├─ eslint.config.js        # ESLint code hygiene rules
├─ package.json            # Script directives and third-party dependencies array
├─ tsconfig.json           # TypeScript settings matching TanStack build requirements
├─ vite.config.ts          # Vite bundler, plugin settings, and tsconfig mappings
├─ public/                 # Static mock assets (icons, maps elements)
├─ src/
│   ├─ components/         # Reusable UI component blocks
│   │   ├─ ui/             # Lower-level Radix components (card, input, select, form etc.)
│   │   └─ Header.tsx      # Common sticky navigation header and page footer panels
│   │       └─ IndiaMap.tsx # Custom SVG map tracking routes between Hyderabad/Bhimavaram
│   ├─ hooks/              # Custom React hook utilities (e.g. use-mobile)
│   ├─ lib/                # Application context, constants, errors, and utility helpers
│   │   ├─ error-capture.ts # Runtime SSR error catching hooks
│   │   ├─ error-page.ts    # Elegant SSR fallback screen rendering
│   │   ├─ ucab-data.ts     # Core pricing definitions, data states, and localStorage hooks
│   │   └─ utils.ts         # Styles merging helper (cn classnames merger)
```

└─ routes/	# Directory-based TanStack routing structure
└─ __root.tsx	# Base template (navbar layout environment router-wrapper)
└─ index.tsx	# Ucab Landing homepage (features showcase, live stats)
└─ book.tsx	# Interactive booking and invoice summary interface
└─ track.tsx	# Live simulation tracking (5-stage ride progress sequence)
└─ history.tsx	# Saved trip history overview loaded from localStorage
└─ login.tsx	# Cellular OTP registration form mock
└─ profile.tsx	# User's travel logs count, overall spending sum, and mock cards
└─ routeTree.gen.ts	# Automated route layout definition map created by TanStack
Router	
└─ router.tsx	# TanStack Router instance creation config
└─ server.ts	# Server entry point wrapper handling Nitro SSR redirects
└─ start.ts	# System start runner
└─ styles.css	# Master stylesheet comprising Tailwind imports and root
overrides	

Application Routing & Flows

The frontend comprises six key user flows, each mapping directly to a TanStack Router file-based route in `src/routes/` :

```
graph TD
    Home["Home (/)"] -->|Book now / Book Ride| Book["Booking (/book)"]
    Home -->|Login Button| Login["Login (/login)"]
    Book -->|Confirms payment| Track["Live Track (/track?id=...)"]
    Track -->|Menu link| History["Ride History (/history)"]
    Track -->|Menu link| Profile["Profile (/profile)"]
    Login -->|Verification success| Book
```

1. Homepage (`/index.tsx`):

- Showcases the service value proposition with localized marketing (e.g., "Rides across India · Pay in ₹").
- Displays static metric counters (e.g. "50k+ Rides Completed", "4.8★ Avg Rating").
- Includes a list of vehicles and approximate fare examples for a 5 km ride.
- Embeds the interactive `IndiaMap` illustrating a sample route snippet: *HITEC City ↔ Bhimavaram Rly Stn*.

2. Ride Booking (`/book.tsx`):

- Enables selection of pickup/drop-off destinations within or across cities.
- Features vehicle category selectors (Mini, Sedan, SUV, Auto) with real-time bill projections.
- Accepts coupon entries (e.g. `UCAB10`) and optional service extras (adds refreshments, charitable donations).
- Embeds a custom SVG Map pinpointing selected coordinates.

3. Simulation Tracking (`/track.tsx`):

- Uses URL search parameters to locate a booking (e.g. `/track?id=UCXXXX`).
- Features a 5-step interval progress engine simulating actual trip progression.
- Simulates driver partner metadata ("Ravi Kumar · TS 09 AB 1234") and provides dummy calling hooks.

4. Ride Log History (/history.tsx):

- Queries `localStorage` to showcase previous trips chronologically.
- Shows ride details like starting point, ending point, date/time, vehicle category, and ride status.

5. User Profile (/profile.tsx):

- Calculates cumulative statistics from the user's ride history (e.g. total rides, aggregate spendings).
- Presents mock options for saved addresses (Home, Work, College) and payment methods (saved cards, wallets).

6. OTP Login (/login.tsx):

- Validates a 10-digit mobile number format.
- Simulates OTP dispatch. Accepts any 4-digit code (noting demo passcode: `1234`) to write user info into `localStorage` and redirects to `/book` .

 Pricing Details & Business Rules

All core data structures and pricing metrics are defined in `src/lib/ucab-data.ts` .

1. Vehicle Categories Model

Fares depend on the selected vehicle type:

Fleet Class	Base Fare (₹)	Rate per Km (₹)	Seats	Default ETA	Emoji	Description
Ucab Auto	₹30	₹10 / km	3	2 min		Quick short rides
Ucab Mini	₹40	₹12 / km	4	3 min		Compact, budget-friendly
Ucab Sedan	₹60	₹16 / km	4	5 min		Comfortable AC sedan
Ucab SUV	₹90	₹22 / km	6	7 min		Spacious for groups

2. Location Geolocation Array (PLACES)

The database defines fixed cities coordinates in Telangana (Hyderabad) and Andhra Pradesh (Bhimavaram):

- **Hyderabad:**
 - *Rajiv Gandhi Intl. Airport* (Shamshabad) - `lat: 17.24, lng: 78.43`
 - *HITEC City* (Madhapur) - `lat: 17.44, lng: 78.38`
 - *Charminar* (Old City) - `lat: 17.36, lng: 78.47`
 - *Secunderabad Rly Stn* (Secunderabad) - `lat: 17.43, lng: 78.50`
 - *Banjara Hills* (Road No. 12) - `lat: 17.41, lng: 78.44`
 - *Gachibowli Stadium* (Gachibowli) - `lat: 17.44, lng: 78.34`
- **Bhimavaram:**
 - *Bhimavaram Town Rly Stn* - `lat: 16.54, lng: 81.52`
 - *SRKR Engineering College* (China Amiram) - `lat: 16.55, lng: 81.51`
 - *Mavullamma Temple* (Juvvalapalem Rd) - `lat: 16.55, lng: 81.53`

- *Bhimavaram Bus Stand (Main Rd)* - lat: 16.55, lng: 81.52
- *Gunupudi Market (Gunupudi)* - lat: 16.54, lng: 81.51

3. Distance Estimation Algorithm

Distance calculation utilizes a coordinate coordinate-vector delta model scaled to approximate Indian geographical standards:

```
export function estimateDistanceKm(from: Place, to: Place): number {
  if (from.city !== to.city) return 380; // Hyderabad ⇄ Bhimavaram approximate flat rate
  const dLat = (to.lat - from.lat);
  const dLng = (to.lng - from.lng);
  const km = Math.sqrt(dLat * dLat + dLng * dLng) * 111;
  return Math.max(2, Math.round(km * 10) / 10);
}
```

4. Fare Calculator Blueprint

Prices are formatted in Indian Rupee denomination:

- **Gross Base Ticket:** Base Fare + (perKm Rate × distanceKm)
- **Discount deduction:** Deduction of 10% calculated against the Base ticket if coupon matches `UCAB10` (case insensitive).
- **GST calculation:** 5% taxation added on top of the discounted rate: $\text{Tax Amount} = \text{Math.round}((\text{Gross Base} - \text{Discount}) \times 0.05)$
- **Total Fare (INR):** $\text{Total} = \text{Max}(0, \text{Math.round}(\text{Gross Base} - \text{Discount} + \text{Tax}))$

5. Booking Local Persistence

Bookings are persisted locally using the client's `localStorage` utilizing standard serialization:

- *Key name:* `ucab_bookings`
- *Write behavior:* Active bookings are prepended (`unshift()`) to keep the history in reverse chronological order.
- Once a simulated trip reaches the "Completed" status in the tracking interface, an update request saves `status: "completed"` .

Development & Script Directives

1. Prerequisites

Ensure you have **Bun** or **Node.js** installed on your developer machine. (Bun is recommended for optimal compatibility).

2. Dependency Installation

Run the following script to install packages:

```
bun install
```

3. Local Development Server

Execute the Vite HMR engine to run Ucab locally:

```
bun run dev
```

Open a browser and navigate to: `http://localhost:3000` (or the console printed port).

4. Code Formatting

Format source files to meet Prettier hygiene requirements:

```
bun run format
```

5. Production Build & Optimization

Bundle optimized assets for deployment:

```
bun run build
```

Preview the static SSR production output:

```
bun run preview
```